

Register New User

We have our `RegisterController` and `register` method that returns a view. Let's add the form to that view. We can use our input components that we used in our other forms.

Open the `resources/views/auth/register.blade.php` file and add the following:

```
<x-layout>
  <div
    class="bg-white rounded-lg shadow-md w-full md:max-w-xl mx-auto mt-12 p-8 py-
12"
  >
    <h2 class="text-4xl text-center font-bold mb-4">Register</h2>
    <form method="POST" action="{{ route('register.store') }}">
      @csrf
      <x-inputs.text id="name" name="name" placeholder="Full name" />
      <x-inputs.text
        id="email"
        name="email"
        type="email"
        placeholder="Email address"
      />
      <x-inputs.text
        id="password"
        name="password"
        type="password"
        placeholder="Password"
      />
      <x-inputs.text
        id="password_confirmation"
        name="password_confirmation"
        type="password"
        placeholder="Confirm Password"
      />
      <button
        type="submit"
        class="w-full bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded
focus:outline-none"
      >
        Register
      </button>

      <p class="mt-4 text-gray-500">
        Already have an account?
        <a class="text-blue-900" href="{{ route('login') }}">Login</a>
      </p>
    </form>
  </div>
</x-layout>
```

We added the input components, we are using the `@csrf` directive to add the CSRF token to the form and we are using the `route` helper to generate the URL for the `action` attribute as well as the link to the login page.

Notice that I used an underscore for `password_confirmation`. It has to be named this if you want to use Laravel's built-in validation rule for password confirmation.

store Method

Now let's handle the `store` method in our `RegisterController`. Open the `app/Http/Controllers/Auth/RegisterController.php` file and add the following imports:

```
use Illuminate\Support\Facades\Hash;
use App\Models\User;
```

Validating the Request

Add the following code to the `store` method:

```
public function store(Request $request): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|string|email|max:255|unique:users',
        'password' => 'required|string|min:8|confirmed',
    ]);

    print_r($validatedData);
    die();
}
```

We are using the `validate` method to validate the incoming request data. If the validation fails, a redirect response will be generated to send the user back to their previous location. The errors will also be flashed to the session so they are available for display. The form should display the errors because of the way we handle the validation errors in the components.

As far as the rules, name must be a string, max 255 characters, email must be a string, email, max 255 characters, unique in the users table, password must be a string, min 8 characters, and it must be confirmed.

Hashing the Password

Before we save the user to the database, we need to hash the password. Laravel provides a helper function called `bcrypt` that we can use to hash the password. Update the `store` method in the `RegisterController` by adding the following line:

```
// Hash the password
$validatedData['password'] = Hash::make($validatedData['password']);
```

We are using the `Hash::make` method to hash the password. Now we can save the user to the database. We will use the `User` model to create a new user. Add the following line to the `store` method:

```
// Create a new user
$user = User::create($validatedData);
```

Redirecting the User

Let's redirect the user to the homepage after they register. Add the following line to the `store` method:

```
return redirect()->route('home')->with('success', 'Registration successful You can now log in!');
```

Now you can register a new user. If you try to register a user with the same email address, you will get an error message. If you register a user successfully, you will be redirected to the homepage with a success message.

In the next lesson, we will add authentication to our application.